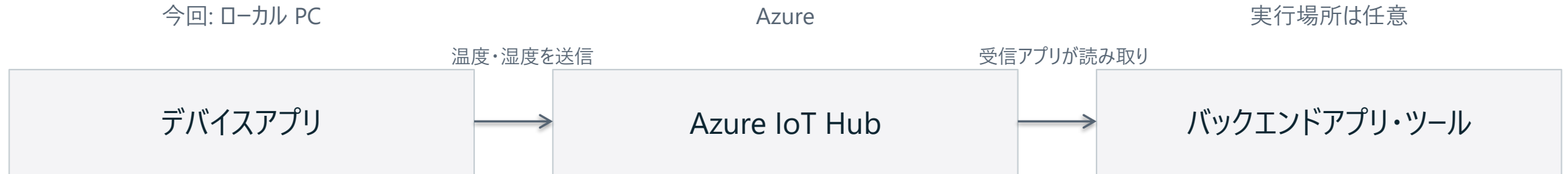


デバイス・IoT Hub・バックエンドの役割

デバイスアプリがデータを送り、IoT Hub が通信を仲介し、バックエンドがデータを利用する。

主体



担当すること

測定値を作って送る。
受け取った操作や設定変更を実行する

デバイスを認証し、
デバイスとの通信と後続への配送を担う

テレメトリを読み、
保存・業務処理を行う。
デバイスへの操作を要求する

保存・分析は今回の受信確認とは別

IoT Hub は接続・通信の基盤であり、長期保存・分析・デバイス上の処理はアプリや後続サービスと組み合わせる（[1]、[2]）。

今回: 温度センサー役のローカル PC → IoT Hub → 受信ツール。

Hub 作成からデバイス接続までの手順

Hub と Device ID を作っても、デバイスアプリが自動で接続するわけではない。

クラウド側の準備

アプリの準備

実行と確認

段階



誰が何をするか

管理者がAzureにIoT Hubを作る

登録担当がHubに IDと認証方式を登録する

開発・設定担当がアプリに 接続先と資格情報を渡す

デバイスアプリが 起動し、 認証を伴う接続を行う

デバイスが送り、受信担当 がツールで読む

この段階で用意・確認できること

接続先のHub

そのHubで受け入れる デバイスの情報

接続に必要な設定

接続処理の結果

対象デバイスのメッセージ

IDレジストリはデバイスIDと認証情報を管理する場所であり、登録しただけではアプリは動作しない（[1]）。

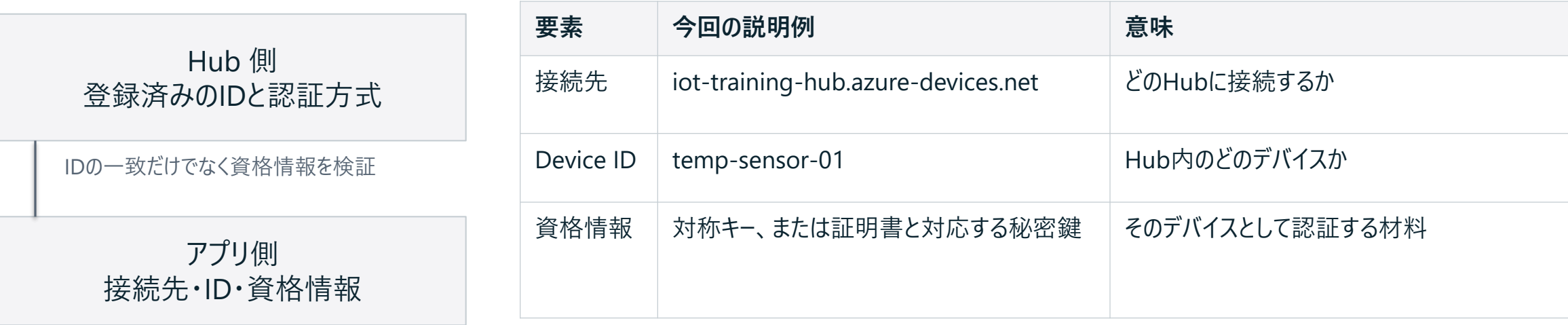
今回は手動登録。多数台の自動プロビジョニングは第7章で扱う。

登録済み ≠ 接続中

[\[1\] ID レジストリ](#)

Device ID とデバイスの資格情報

Device ID は名前。資格情報は、そのデバイスとして認証するために使う。



デバイス認証の入口は、**対称キーで生成する有効期限付きのSASTークンとX.509形式のデジタル証明書**。
IDを知っているだけでは認証できない（[1]、[2]）。

今回の対称キー方式の接続文字列の形（説明用・接続不可）：

```
HostName=iot-training-hub.azure-devices.net;  
DeviceId=temp-sensor-01;  
SharedAccessKey=<非表示>
```

実際は1行で設定

接続文字列はURLではなく、キーを含む秘密情報。画面・コード・ログに公開しない。（[1]）

[\[1\] SAS による認証](#)

[\[2\] X.509 による認証](#)

認証・権限・通信保護の違い

誰として接続するか、何を許可するか、通信内容をどう守るかは別の問い。

観点	確認する問い	具体例
認証	誰として接続するか	temp-sensor-01 として資格情報を検証する
認可・権限	その主体に何を許すか	自分のテレメトリ送信と、サービス側からの操作を分ける
通信保護	経路上で内容を守れるか	TLSで通信を暗号化する



デバイス側は **SAS / X.509**。

デバイス管理・操作を行うサービスAPIは **Entra IDで認証 + Azure RBACで権限付与**、または **共有アクセスポリシー（Hub側の権限設定）** のSASを使う。

Entra ID認証はデバイスAPIには使えない（[1]、[2]）。

デバイス用とサービス側の資格情報を分ける。TLSを使っている、キーの公開が安全になるわけではない。（[3]、[4]）

[1] SAS とアクセス権

[3] TLS サポート

[2] Microsoft Entra ID によるサービス API の認証

[4] セキュリティ推奨事項

IoT Hub の通信プロトコルと選び分け

通信プロトコルはデータをやり取りする方式。デバイスの制約と受信方法に合わせて選ぶ。

観点	MQTT	AMQP	HTTPS
主な選択場面	一般的なデバイスの直接接続	複数デバイスを集約するゲートウェイ	他の方式を利用できないデバイス
クラウド→デバイスの メッセージ受信（C2D）	サーバーからプッシュ	サーバーからプッシュ	デバイスから定期的に問い合わせる （ポーリング）
1つのTLS接続で扱うデバイスID	1つ	複数IDを多重化できる	1つ
送信先ポート	8883 / WebSocket経由は443	5671 / WebSocket経由は443	443

IoT Hubのデバイス側通信を比較。C2Dメッセージを速やかに受け取る用途はMQTT / AMQPを選ぶ。操作機能全般の対応比較ではない（[1]）。

443番ポートを使う通信が、すべてHTTPS方式になるわけではない。MQTT / AMQPはWebSocket経由でも利用できる（[2]）。

IoT HubのMQTT対応は機能が限定され、汎用MQTTブローカーと同じではない。（[3]）

[1] 対応プロトコル・選択基準・ポート

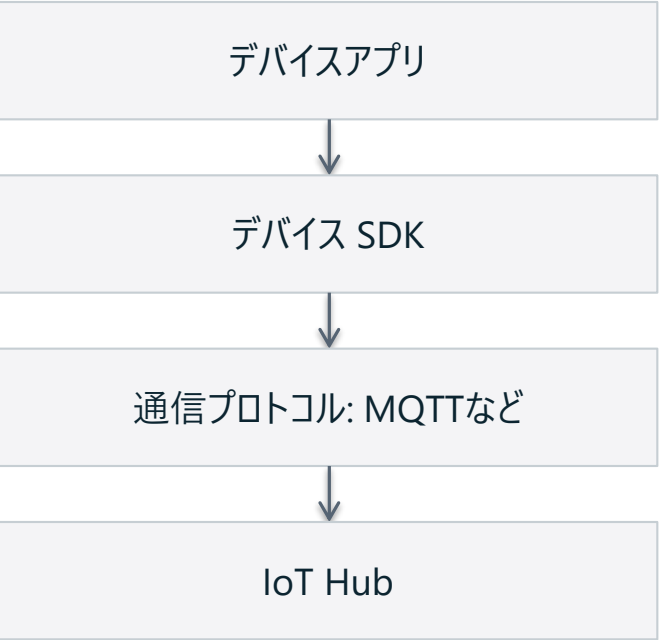
[2] WebSocketとポート番号

[3] MQTTサポートの範囲

SDK の役割と用途別の使い分け

プロトコルは通信の方式。SDKは、その通信機能をアプリから使うためのライブラリ。

前ページの通信方式をアプリから利用する際は、SDKの対応範囲も確認する。SDKごとに使えるプロトコルは異なる（[1]）。



やりたいこと	使うライブラリ
デバイスから送信し、指示を受ける	IoT Hub デバイス SDK
デバイスIDを管理し、操作や設定変更を要求する	IoT Hub サービス SDK
Hubリソース自体を作成・管理する	IoT Hub 管理 SDK
組み込みエンドポイントからD2Cを読む	Event Hubs SDK

読み取り口は03-08で確認

SDKは実行する役割に合わせて選ぶ。D2Cはデバイスからクラウドへの通信（Device-to-Cloud）を指す（[2]、[3]）。

[\[1\] 通信プロトコルの選択](#)
[\[3\] D2C 読み取りの SDK](#)

[\[2\] IoT Hub SDK](#)

温度テレメトリの本文と送信処理

テレメトリは、測定値やイベントをデバイスから送るデータ。本文はデバイスアプリが作る。([1])

本文の説明例（この講義では温度は摂氏、湿度は相対湿度%）：

```
{"temperature":25,"humidity":45}
```

temperature: 摂氏 / humidity: 相対湿度%



1. アプリが温度・湿度の値を用意する。
2. JSONの本文をメッセージにし、デバイスSDKで送る。
3. IoT Hubを経由して、受信側が読み取る。

今回の実装はランダム値を使った模擬センサー。起動・接続だけでは送信せず、start の操作後に送信する。([2])

本文の項目名・単位・意味はアプリ側で決める。IoT Hubへの送信だけで分析が完了するわけではない。([3]、[1])

[1] IoT Hub のメッセージング

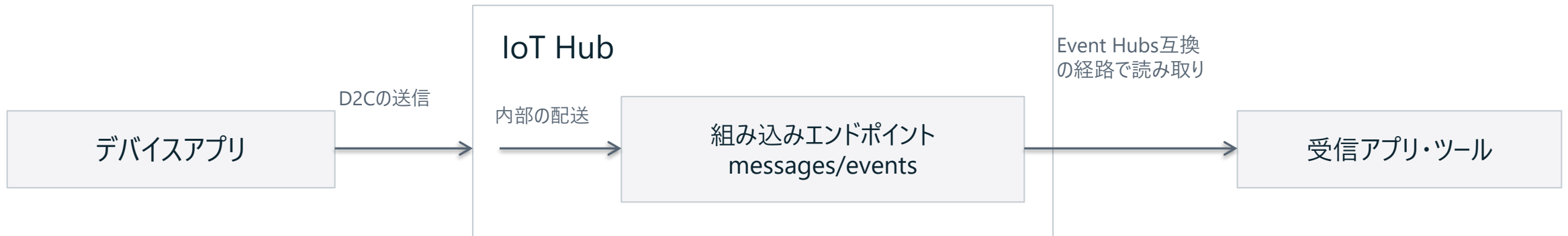
[3] メッセージの構造

[2] デモ実装

D2C を読む組み込みエンドポイント

受信アプリは、IoT Hubに用意されたD2Cの読み取り口を使える。

ここではルート未追加で、既定の組み込み宛て配送を有効にした構成を前提にする。配送設定を変更した場合の条件は第5章で扱う。



組み込みエンドポイント messages/events は **Event Hubs互換**。対応するSDKで読み取れる。

この経路の利用に、別のEvent Hubsリソースの作成は不要。

組み込みエンドポイントは期限付きでメッセージを保持する読み取り口であり、長期保存用のDBではない（[1]）。

この読み取り口は共有アクセスキー方式で接続する。サービスAPIのEntra ID認証とは別。デバイス用の接続文字列は使わない。（[2]、[3]）

[1] 組み込みエンドポイントの読み取り・配送・保持

[3] 接続に必要な情報と権限

[2] 組み込みエンドポイントの認証制約

送信ログと受信データの照合

送信側の成功ログと、受信側に現れた送信元・本文を対応させて確認する。

送信側の表示例（説明用）：

受信側で見る項目（ツール固有の出力形式ではなく、説明用に整形）：

```
Device connected to IoT Hub.  
Sent telemetry: {'temperature': 25, 'humidity': 45}
```

確認項目	表示例
送信元デバイスのメタデータ	temp-sensor-01
メッセージ本文	{"temperature":25,"humidity":45}

送信元デバイスの情報と、アプリが作ったJSON本文は別の情報。IoT Hubが付ける送信元IDを含むシステムプロパティと本文を区別する（[1]）。

送信処理の成功

受信側で確認

保存・業務処理: 未確認

送信ログだけでは受信ツールでの確認やDB保存の完了は分らない。
受信表示を確認しても、後続の業務処理完了は別に確認する。（[2]、[3]）

この値の照合は説明用。同じ値の繰り返しがあり得るため、1件を一意に識別した証明ではない。

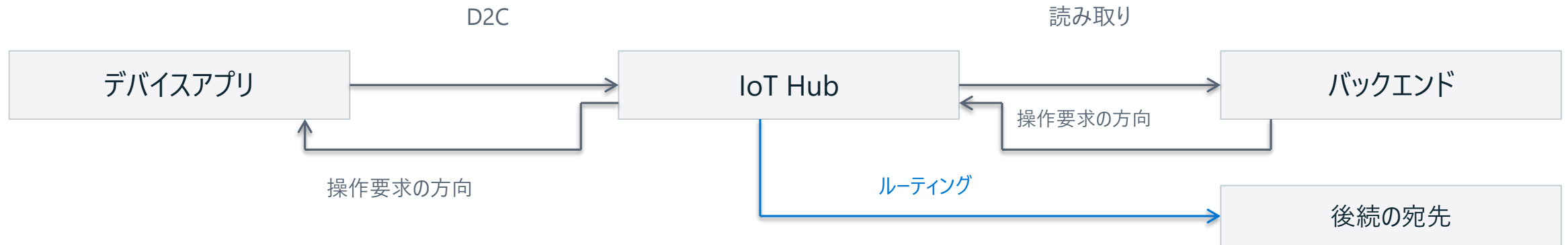
[1] メッセージの構造とシステムプロパティ

[2] 既存実装のログ出力位置

[3] 組み込みエンドポイントからの読み取り

デバイス操作・設定変更・後続配送

温度を集めるD2Cに加えて、操作・状態同期・後続配送の機能がある。



追加したい要求	使う機能	詳細を扱う章
送信を開始・停止したい	Direct Method: デバイス上の処理を呼び出す	第4章
送信間隔を変え、適用状態を知りたい	Device Twin: 期待する設定と報告された状態を扱う	第4章
デバイスへ通知・依頼を届けたい	C2D (Cloud-to-Device) メッセージ: クラウドからデバイスへ送る	第4章
条件に応じて保存先へ渡したい	メッセージルーティング: 後続の宛先へ配送する	第5章

これらはIoT Hubの異なる機能。操作や設定の実際の適用処理はデバイスアプリ側に必要（[1]、[2]）。

Basic・Standard・Free の機能差

使う機能を選ぶことと、必要な容量を見積もることは別。

今回の機能を利用可能: Standard / Free

今回確認する機能	Basic	Standard	Free（評価用）
D2C・メッセージルーティング	対応	対応	対応
Direct Method・Device Twin・C2D	非対応	対応	対応
位置づけ	データ収集中心	双方向通信・デバイス管理	Standard相当の機能を小容量で評価

この表は講義で使う機能に絞った比較。BasicとStandardの認証・セキュリティ機能は同じ。
Freeは評価用で容量が限られ、Basic / Standardへ直接アップグレードできない。（[1]）

今回の開始・停止と設定変更には、Standard相当の機能が必要。Basicでは本講義の双方向デモを実施できない。（[1]）

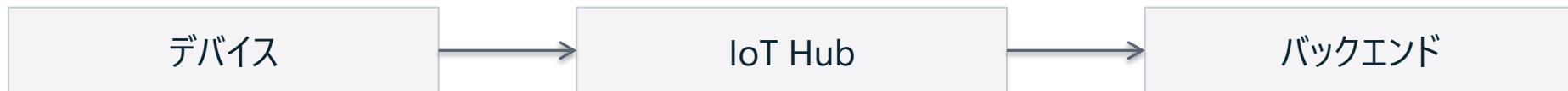
台数 × 送信頻度 × メッセージサイズ → 容量設計は第7章

[\[1\] レベル別の機能と Free の制約](#)

登録・接続・送受信の確認問題

次の3つの場面で、何が不足しているか説明してください。

1. Hubに temp-sensor-01 を登録しました。アプリにはDevice IDだけ渡せば接続できますか。
2. デバイス側に送信ログが出ました。受信担当はどこを読み、何を照合しますか。
3. 受信ツールで温度を確認できました。DB保存は完了していますか。
次に送信を停止したい場合、誰が要求し、誰が実行しますか。



次章: Direct Method・Device Twin・C2Dを、操作と状態同期の用途で使い分ける。